



**University of Engineering and Technology,
Lahore**

Department of Computer Science

Project Proposal

Team Members

Name	Roll Number	Email
Muhammad Uzair	2026(S)-CS-96	ranauzair32303@gmail.com
Session: Spring'26 Morning	Section: B	Semester: 1 st

Course: CSC101 - Discrete Mathematics

Teacher: Mr. Waqas Ali

May 31, 2026

Project Proposal

Submission Deadline: Monday, 31-5-2026, 11:59 PM

1. Project Title

Recursive Algorithm Tracer: Visualising How Recursion Works Step by Step

2. Selected Track

- Track 1: Foundations of Logic (Rosen Ch. 1–2)
- Track 2: Discrete Structures (Rosen Ch. 2, 9)
- **Track 3: Algorithm Design & Analysis (Rosen Ch. 3)**
- Track 4: Graph & Tree Algorithms (Rosen Ch. 10)
- Track 5: Number Theory Applications (Rosen Ch. 4–5)
- Track 6: Counting & Combinatorics (Rosen Ch. 6)

3. Chosen Project

Recursive Algorithm Tracer , a console application that implements five classic recursive algorithms (Factorial, Fibonacci, GCD, Power, and Tower of Hanoi) and prints a live, indented call-depth trace for each one so the user can follow exactly how recursion unwinds. Every run is also saved to a log file (recursion_log.txt).

4. Problem Statement

Recursion is one of the most important ideas in computer science, yet it is hard to understand from code alone because the call stack is invisible. The core challenge is to make the recursive process visible: show every function call as it happens, indent it according to how deep in the stack it is, and show the return value as each call unwinds. Without this kind of trace, beginners often struggle to see how recursive problems are broken into smaller sub-problems and how the base case stops infinite calling.

5. Motivation

Recursion is covered in Rosen Chapter 5 under recursive definitions and structural induction. The algorithms chosen ,Factorial, Fibonacci, GCD, Power, Tower of Hanoi, are the exact examples used in the textbook. Building a tracer that mirrors the book's diagrams in the terminal makes the theory concrete. The file-logging feature also gives a record to study later, which is useful when preparing for exams.

6. Mathematical Foundations

- **Recursive definitions (Rosen Ch. 5.3):** Each algorithm is defined by a base case and a recursive step, exactly the structure of a recursive definition.
- **Factorial:** $n! = n * (n-1)!$, base case $0! = 1$. Grows as $O(n)$.
- **Fibonacci:** $F(n) = F(n-1) + F(n-2)$, base cases $F(0)=0$, $F(1)=1$. Each call branches into two sub-calls.
- **GCD , Euclidean Algorithm (Rosen Ch. 4.3):** $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, base case $\text{gcd}(a, 0) = a$.
- **Power:** $\text{base}^{\text{exp}} = \text{base} * \text{base}^{(\text{exp}-1)}$, base case $\text{base}^0 = 1$.
- **Tower of Hanoi (Rosen Ch. 5.3):** $T(n) = 2 * T(n-1) + 1$, which solves to $2^n - 1$ total moves. Proved by induction.

7. Planned Approach

- **Programming language:** C++ only basic features: functions, loops, if-else, arrays, file I/O with `<fstream>`, and console I/O with `<iostream>`.
- **Main algorithms:** Each of the five recursive functions takes an extra 'depth' parameter. At each call the function prints its own indentation (depth * two spaces + a bar), then calls itself with depth+1. On return it prints the computed value.
- **Key features:** (1) Indented call trace with depth bars so nesting is visible; (2) Return-value annotation as each call unwinds; (3) File logging , every line printed to the screen is also written to `recursion_log.txt` using `ofstream`.
- **Testing:** Run each algorithm on small inputs whose answers are known (e.g., $5!=120$, $F(6)=8$, $\text{gcd}(48,18)=6$, $2^5=32$, $\text{Hanoi}(3)=7$ moves) and verify the trace count matches the expected number of calls.

8. Expected Outcomes

The completed program will: (1) display a fully indented call trace for each selected algorithm; (2) print the return value at every level so the unwinding process is clear; (3) confirm the final answer matches the formula (e.g., $2^n - 1$ for Hanoi); (4) save the complete trace to `recursion_log.txt` so it can be reviewed after the program closes.

9. Potential Challenges

- **Deep Fibonacci traces:** $F(8)$ already makes over 40 calls. The solution is to cap the input at a small number ($n \leq 8$) with a clear warning.
- **Integer overflow for large factorials:** Factorial grows very fast. Using long long handles up to $12! = 479,001,600$ safely.
- **File I/O errors:** Will check that the log file opens successfully before writing and report an error if it cannot.

10. Resources Required

- C++ compiler (g++) , no external libraries.
- Standard headers `<iostream>` and `<fstream>` only.
- Rosen's textbook, Chapters 4.3 and 5.3, for definitions and examples.

11. References

1. Rosen, K. H. *Discrete Mathematics and Its Applications*. Chapters 4.3 (GCD) and 5.3 (Recursive Definitions, Factorial, Fibonacci, Hanoi).
 2. C++ reference — cppreference.com (`std::ofstream`, basic I/O).
 3. Cormen et al. *Introduction to Algorithms* — Chapter 4, for recursive algorithm analysis background.
-

Declaration: We confirm that this proposal represents our own work and that we understand the academic integrity requirements outlined in the project guidelines.

Signatures:

Member 1: __Muhammad Uzair__

Date:

30/05/2026